Glowy

Dossier P1 : Comptes, sécurité et Super Admin

Projet d'équipe · octobre 2026

Ce dossier explique le projet en détail, puis **ta gestion** : ce qu'elle fait, ses règles métier, son IA, les fichiers à connaître et comment la présenter à la validation.

Le projet Glowy en détail

1. Qu'est-ce que Glowy ?

Glowy est une **plateforme de réservation de salons de beauté vérifiés en Tunisie** (coiffure, barbier, ongles, spa, visage…). Une cliente cherche un salon, voit les prestations, les prix en dinars, les avis et l'affluence, puis réserve un créneau précis en moins de 2 minutes. Le salon gère tout son travail depuis un tableau de bord, et un Super Admin contrôle la plateforme.

- **4 rôles** : cliente, employé, gérant de salon, Super Admin.
- **3 langues** : français, arabe (écriture de droite à gauche) et anglais.
- **Application installable** sur téléphone (PWA), utilisable hors connexion.
- **5 intelligences artificielles**, une par membre de l'équipe.
- Projet d'équipe de 5 étudiants : chaque personne a **1 gestion classique + 1 IA**.

2. Les technologies

Domaine	Technologie	Rôle dans le projet
Langage	**TypeScript**	partout : pages, API, règles métier, tests
Framework	**Next.js 16** (App Router) + **React 19**	site et API dans un seul projet
Interface	**Tailwind CSS 4**, composants shadcn / Base UI	mise en page responsive, mobile first
Animations	**GSAP**	apparitions et transitions
Base de données	**MongoDB** + **Mongoose 9**	33 collections (comptes, salons, réservations, avis, points…)
Authentification	**Auth.js v5**	connexion e-mail / mot de passe, Google, sessions sécurisées
Validation	**Zod**	contrôle de toutes les données reçues par l'API
Carte	**Leaflet** + OpenStreetMap	carte des salons

Domaine	Technologie	Rôle dans le projet
Graphiques	**Recharts**	statistiques du dashboard
E-mails	**nodemailer** (SMTP)	mot de passe oublié, rappels
Notifications	**Web Push**	rappels sur le téléphone
Tests	**node:test** + tsx	75 tests automatisés sur une base de test

3. L'architecture

Le code suit une **architecture propre (Clean Architecture)** : chaque domaine est un **module** indépendant dans `src/modules/<module>/`, découpé en 3 couches.

Couche	Contenu	Exemple
`domain`	règles métier pures, sans base de données, faciles à tester	calcul de l'acompte, verrou des créneaux
`application`	cas d'usage et validation des données (Zod)	« créer une réservation »
`infrastructure`	lecture et écriture dans MongoDB	requêtes Mongoose

Organisation des dossiers :

```
src/
  app/              pages et routes API (Next.js)
    (site)/         site public + espace client (/compte)
    (auth)/         connexion, inscription, mot de passe oublié
    dashboard/      espace du salon (gérant, employés)
    admin/          espace Super Admin
    api/            routes REST
  modules/          un dossier par domaine (identity, booking, reviews, loyalty…)
  components/       composants d'interface
  core/domain/      règles communes (erreurs, heure de Tunis, événements)
  infrastructure/   connexion MongoDB, sessions, outils HTTP
  i18n/             traductions français / arabe / anglais
public/sw.js        service worker (hors connexion, notifications)
tests/              tests automatisés
```

Les modules communiquent par événements. Exemple : quand le salon marque un rendez-vous « terminé », le module réservation publie `booking.completed`. Le module fidélité y réagit en créditant les points, et le module notifications prévient la cliente. Aucun module ne modifie le code d'un autre.

Tâches planifiées (toutes les 10 minutes) : rappels de rendez-vous, préavis et expiration des points, analyse des nouveaux avis.

4. La base de données

MongoDB stocke les données sous forme de documents. Les données qui vivent toujours avec leur parent sont **imbriquées** : les horaires et les prestations dans le salon, les lignes et le paiement dans la réservation, le compte fidélité dans l'utilisateur.

Collection	Contenu
`User`	comptes (rôle, mot de passe haché, points, favoris)

Collection	Contenu
`Salon`	salons (adresse, horaires, photos, prestations, plan)
`Employee`, `TimeOff`	équipe et congés
`Booking`	réservations (prestations, statut, acompte)
`SlotLock`	verrou anti double réservation (tranches de 5 min)
`Review`, `ReviewReport`	avis et signalements
`PointTransaction`, `Reward`, `Redemption`	fidélité
`Notification`, `PushSubscription`	notifications
`PlanPayment`	abonnements des salons
`AuditLog`	journal des actions sensibles

5. La sécurité

- Mots de passe hachés (bcrypt) ; compte bloqué 15 minutes après 5 échecs.
- Rôle vérifié sur **chaque page et chaque API**.
- Protection CSRF : une écriture venant d'un autre site est refusée.
- Toutes les données reçues sont validées (Zod) avant d'arriver en base.
- Les actions sensibles du Super Admin sont journalisées.

6. L'application mobile (PWA)

- **Manifest** : nom, icônes, mode plein écran, raccourcis. L'application s'installe sur l'écran d'accueil.
- **Mode hors connexion** : le service worker enregistre les pages consultées et précharge les pages publiques liées. Sans réseau, elles s'ouvrent depuis le téléphone ; sinon la page `/offline` liste les pages disponibles. Les pages personnelles sont effacées à la déconnexion.
- **Mobile first et responsive** : testé de 320 à 1440 px de large, en français, en arabe et en anglais, sans aucun débordement.

7. Lancer le projet

```
npm install
npm run db:seed        données de démonstration
npm run dev            développement : http://localhost:3000
npm run build && npm start      production (nécessaire pour le hors connexion)
npm test               75 tests automatisés
```

Sur la page de connexion, les boutons **« Accès démo par rôle »** connectent en un clic : Client, Gérant, Employé, Super Admin.

8. Les 5 gestions

#	Gestion	Responsable	IA associée
1	Comptes, sécurité et Super Admin	P1	Recommandation de salons
2	Dashboard du salon	P2	Prédiction d'affluence

#	Gestion	Responsable	IA associée
3	Recherche et réservation	P3	Assistante de réservation
4	Avis, modération et support	P4	Analyse des avis
5	Espace client, fidélité, abonnements et PWA	P5	Analyse beauté par photo

Un parcours complet relie les 5 gestions :

1. La cliente se connecte et voit des salons recommandés (gestion 1).
2. Elle réserve un créneau (gestion 3).
3. Le salon voit le rendez-vous dans son agenda et le marque terminé (gestion 2).
4. La cliente gagne des points (gestion 5) et laisse un avis vérifié, analysé par l'IA (gestion 4).
5. Le Super Admin suit l'activité et modère (gestion 1).

1 Gestion des comptes, de la sécurité et du Super Admin (P1)

But : permettre à chacun de créer un compte et de se connecter en sécurité, et donner au Super Admin le contrôle de toute la plateforme.

Fonctionnalités

Comptes

- Inscription d'une cliente (compte actif tout de suite), avec code de parrainage facultatif.
- Inscription d'un salon avec justificatif. Le salon reste **en attente** jusqu'à sa validation par le Super Admin et n'apparaît pas dans la recherche avant.
- Connexion par e-mail et mot de passe, ou **avec Google**. Une connexion démo par rôle est disponible si `DEMO_LOGIN=true`.
- Mot de passe oublié : un lien valable **1 heure**, à usage unique.
- Vérification de l'adresse e-mail : un lien valable **48 heures**.
- Changement du mot de passe depuis son compte.
- Comptes des employés créés par le gérant, avec un mot de passe temporaire.

Sécurité

- Mots de passe hachés avec bcrypt.
- **Blocage 15 minutes après 5 échecs** de connexion.
- Contrôle du rôle sur chaque page (`src/proxy.ts`) **et** sur chaque API.
- Protection CSRF : une écriture venant d'un autre site est refusée.
- Les liens envoyés par e-mail utilisent `APP_URL`, pour ne pas être piégés par l'en-tête `Host`.
- Au plus un e-mail de lien par minute et par compte.

Super Admin (`/admin`)

- **Vue globale** : salons, clientes, réservations, revenus, courbes sur 12 mois.
- **Salons** : validation ou refus, suspension et réactivation, consultation du dashboard d'un salon.
- **Utilisateurs** : liste, blocage et déblocage, envoi d'un lien de réinitialisation.
- **Journal d'audit** de toutes les actions sensibles.
- **Rapports** par ville, prestation et salon, avec export.
- **Catalogue global** : villes, catégories, langues ; taux de commission ; **annonces** envoyées à tous.
- **Barème de fidélité** et campagnes « double points » (le moteur de points relève de la gestion 5).
- **Boîte d'envoi** : e-mails et SMS envoyés ou simulés.

IA — Recommandation de salons

- **Entrées** : rendez-vous terminés (catégories, prestations, ville, budget), salons favoris, note des salons, disponibilités du jour.
- **Méthode** : un profil de goûts est construit à partir de l'historique. Chaque salon reçoit un **score** qui combine :
  - la ressemblance avec ce profil ;
  - sa qualité, calculée par une **note bayésienne** : un salon avec peu d'avis est ramené vers 4/5 ;
  - ses créneaux libres aujourd'hui.

- **Sortie** : une liste classée, avec **la raison affichée** (« même catégorie que vos visites », « bien noté », « disponible aujourd'hui »…).
- **Repli** : sans historique (visiteur ou nouveau compte), ce sont les salons les plus populaires.
- **Code** : `src/modules/catalog/domain/recommend.ts` · `GET /api/ai/recommendations`.

Repères techniques

Modules	`identity`, `admin`, `platform`
Pages	`/connexion`, `/inscription`, `/mot-de-passe-oublie`, `/reinitialiser`, `/verifier-email`, `/admin/*`
API	`/api/auth/*`, `/api/me/password`, `/api/me/verify-email`, `/api/admin/*`, `/api/ai/recommendations`
Collections	`User`, `AccountToken`, `AuditLog`, `PlatformSetting`, `Announcement`
Événements émis	`user.registered`, `salon.registered`, `salon.statusChanged`
Tests	`oauth`, `password-reset`, `platform`, `recommend`, `loyalty-admin`

Ma préparation pour la validation

Fichiers à connaître

- `src/modules/identity/` (comptes, connexion, mot de passe), `src/modules/admin/`, `src/modules/platform/`
- `src/auth.ts`, `src/auth.config.ts`, `src/proxy.ts` (contrôle des rôles)
- Pages : `src/app/(auth)/*`, `src/app/admin/*` · API : `src/app/api/auth/*`, `src/app/api/admin/*`
- IA : `src/modules/catalog/domain/recommend.ts` · tests : `tests/recommend.test.ts`, `tests/password-reset.test.ts`, `tests/oauth.test.ts`

Démo (3 minutes)

1. **Inscription** d'une cliente puis connexion ; montrer le blocage après 5 mauvais mots de passe.
2. **Mot de passe oublié** : demande du lien, lien visible dans `/admin/messages` (ou reçu par e-mail si SMTP configuré).
3. **CRUD Super Admin** :
   - `/admin/utilisateurs` : liste, recherche, pagination, bloquer / débloquer (Read, Update) ;
   - `/admin/salons` : valider, suspendre, réactiver un salon (Read, Update) ;
   - `/admin/fidelite` : campagne « double points » créée, modifiée, supprimée (CRUD complet).
4. `/admin/audit` : chaque action sensible est journalisée.

Montrer l'IA

- Où la montrer : accueil et `/compte` (« Recommandé pour vous »), tri « Recommandé » sur `/salons`.

Règles métier à savoir expliquer

- Mots de passe hachés (bcrypt) ; blocage 15 min après 5 échecs ; compte bloqué = accès coupé aussitôt.
- 4 rôles (cliente, employé, gérant, Super Admin) vérifiés sur chaque page **et** chaque API.
- Un salon reste « en attente » et invisible dans la recherche tant que le Super Admin ne l'a pas validé.
- Lien de réinitialisation valable 1 h, à usage unique, au plus un par minute.

Questions probables du jury

- *Comment empêchez-vous un client d'ouvrir le dashboard ?* `proxy.ts` redirige selon le rôle, et chaque API appelle `requireUser(rôle)`.
- *Pourquoi une note bayésienne ?* Un salon avec 1 seul avis à 5/5 ne doit pas passer devant un salon à 4,8 avec 60 avis.